

# gemstone-py Setup Guide

Installation, environment, and first successful login

Generated from the Markdown sources in gemstone-py/docs  
Assets are local SVG illustrations stored in the repository.

# Contents

## Setup Guide

- What You Need

- Install the Package

  - Which Install Path Should I Use?

  - Installed Package

  - Source Checkout

- Required Environment Variables

- First Real Login

- Recommended First Commands

- GemStone-Side Bootstrap

- Transaction Policy: Read This Early

- First Useful Persistent Write

- First Async Check

- Typed and Lifetime Examples

- Running the Tests

- Troubleshooting

  - ``GemStoneConfigurationError``

  - ``OSError`` while loading the client library

  - Login works in one shell but not another

  - Your writes "worked" but the data is missing

  - A Flask request wrote partial data after a handled error

  - The native backend is not selected

- Where to Go Next

# Setup Guide

---

This guide gets you from "I cloned the repo" to "I can log in to GemStone from Python and run the examples without muttering at the terminal."

## What You Need

At minimum:

- Python 3.11 or newer
- access to a GemStone/S 64 stone
- the GemStone client library ( `libgcirpc` )
- a username and password that can log into the stone

For the full local development experience, you will also want:

- the repository checkout
- a working virtual environment
- the ability to run live tests against your stone
- if you use the self-hosted workflows, a GitHub Actions runner on the GemStone host

## Install the Package

### Which Install Path Should I Use?

| Use case                                 | Command                                                                |
|------------------------------------------|------------------------------------------------------------------------|
| Normal users                             | <code>python3 -m pip install gemstone-py</code>                        |
| Native acceleration                      | <code>python3 -m pip install "gemstone-py[fast]"</code>                |
| Litestar web apps                        | <code>python3 -m pip install "gemstone-py[litestar]"</code>            |
| Source checkout examples/<br>development | <code>python3 -m pip install -e ".[examples,dev]"</code>               |
| VS Code users                            | <code>code --install-extension unicompute.gemstone-py-workbench</code> |

## Installed Package

From PyPI:

```
python3 -m pip install gemstone-py
```

If you want the optional PyO3 native fast path and a wheel exists for your platform:

```
python3 -m pip install "gemstone-py[fast]"  
python -c "from gemstone_py import _gci; print(_gci.IMPLEMENTATION)"
```

From a repository checkout, the fuller backend example is `python -m examples.native_backend.check_backend`.

For web examples from an installed environment:

```
python3 -m pip install "gemstone-py[fastapi]"  
gemstone-fastapi-example --reload
```

For Litestar applications:

```
python3 -m pip install "gemstone-py[litestar]"
```

## Source Checkout

From a repo checkout:

```
python3 -m venv .venv  
source .venv/bin/activate  
python -m pip install -U pip  
python -m pip install -e .[dev]
```

For source-checkout examples without the full development toolchain:

```
python -m pip install -e ".[examples]"  
python -m examples.fastapi.run --reload
```

## Required Environment Variables

The package expects its runtime configuration from environment variables. The minimum set for most local work is:

```
export GS_LIB=/opt/gemstone/product/lib
export GS_STONE=gs64stone
export GS_STONE_NAME=gs64stone
export GS_USERNAME=DataCurator
export GS_PASSWORD=swordfish
```

`GS_STONE` is the canonical stone variable. `GS_STONE_NAME` is accepted as an alias when `GS_STONE` is absent; setting both to the same value keeps tools that use either spelling in sync.

Common optional variables:

```
export GS_HOST=localhost
export GS_NETLDI=netldi
export GS_GEM_SERVICE=gemnetobject
export GS_HOST_USERNAME=
export GS_HOST_PASSWORD=
export GS_LIB_PATH=/full/path/to/libgcirpc-3.7.x-64.dylib
```

Quick reference:

| Variable                      | Required | Example                                      | Purpose                                                                        |
|-------------------------------|----------|----------------------------------------------|--------------------------------------------------------------------------------|
| <code>GS_LIB</code>           | Yes      | <code>/opt/gemstone/<br/>product/lib</code>  | GemStone <code>lib/</code> directory for library discovery                     |
| <code>GS_STONE</code>         | Yes      | <code>gs64stone</code>                       | Stone name                                                                     |
| <code>GS_STONE_NAME</code>    | No       | <code>gs64stone</code>                       | Stone-name alias used by some tools; used when <code>GS_STONE</code> is absent |
| <code>GS_USERNAME</code>      | Yes      | <code>DataCurator</code>                     | GemStone login username                                                        |
| <code>GS_PASSWORD</code>      | Yes      | <code>swordfish</code>                       | GemStone login password                                                        |
| <code>GS_LIB_PATH</code>      | No       | <code>/full/path/<br/>libgcirpc.dylib</code> | Pin to a specific <code>libgcirpc</code> file                                  |
| <code>GS_HOST</code>          | No       | <code>localhost</code>                       | Remote stone host                                                              |
| <code>GS_NETLDI</code>        | No       | <code>netldi</code>                          | NetLDI service name                                                            |
| <code>GS_GEM_SERVICE</code>   | No       | <code>gemnetobject</code>                    | Gem service name                                                               |
| <code>GS_HOST_USERNAME</code> | No       |                                              | Remote host OS username                                                        |
| <code>GS_HOST_PASSWORD</code> | No       |                                              | Remote host OS password                                                        |

# First Real Login

The simplest sanity check is a tiny Python script:

```
from gemstone_py import GemStoneConfig, GemStoneSession

config = GemStoneConfig.from_env()

with GemStoneSession(config=config) as session:
    print(session.eval("1 + 2"))
```

If that prints `3`, the package can:

- read your configuration
- load `libgcirpc`
- log in to GemStone
- evaluate Smalltalk in the repository

That is enough to start.

## Recommended First Commands

The installed package ships with a few small CLI helpers:

```
gemstone-hello
gemstone-smalltalk-demo
gemstone-examples hello
gemstone-examples smalltalk-demo
gemstone-bootstrap --status
gemstone-benchmarks --help
```

From a repository checkout, these examples are also useful first checks:

```
python -m examples.native_backend.check_backend
```

What they are good for:

- `gemstone-hello`

Tiny smoke check for the installed package.

- `gemstone-smalltalk-demo`

Basic bridge demo that is easy to reason about.

- `gemstone-examples ...`

A stable wrapper for the example entry points.

- `gemstone-bootstrap --status`

Checks whether the GemStone-side helper roots are already present.

- `gemstone-benchmarks`

The maintained benchmark lane, distinct from the teaching examples.

- `examples.native_backend.check_backend`

Shows whether the current process selected ctypes or the optional native GCI backend.

## GemStone-Side Bootstrap

Most helpers create their own GemStone-side roots lazily, but an explicit bootstrap step is useful for new stones, shared demo stones, and onboarding checks. The packaged command ships a small Smalltalk file and evaluates it once against the configured stone:

```
gemstone-bootstrap --status
gemstone-bootstrap --dry-run
gemstone-bootstrap
```

The command creates missing helper roots and writes a bootstrap version marker:

| Key                                     | Class                                 | Used by                                       |
|-----------------------------------------|---------------------------------------|-----------------------------------------------|
| <code>GemstonePyBootstrapVersion</code> | <code>String</code>                   | Bootstrap/version audit                       |
| <code>GStoreRoot</code>                 | <code>StringKeyValueDictionary</code> | <code>gemstone_py.gstore.GStore</code>        |
| <code>GSQueryRoot</code>                | <code>Dictionary</code>               | <code>gemstone_py.gsquery.GSCollection</code> |

It does not replace existing helper roots. If your stone already has `GStoreRoot` or `GSQueryRoot`, the command reports them as present and leaves them untouched. `ObjectLogEntry` `objectLog` is checked as a GemStone built-in object log, not created by the bootstrap script.

## Transaction Policy: Read This Early

The most important behavioural rule in `gemstone-py` is that transaction intent should be explicit.

`GemStoneSession(...)` defaults to manual transaction control.

That means:

- a plain session does not silently commit just because your `with` block ended
- write scripts must call `commit()` explicitly or use a scoped helper that commits on success
- read-only scripts should usually abort or use an abort-on-exit policy

The common options are:

```
from gemstone_py import GemStoneSession, TransactionPolicy

with GemStoneSession(
    config=config,
    transaction_policy=TransactionPolicy.COMMIT_ON_SUCCESS,
) as session:
    ...
```

Or:

```
from gemstone_py import session_scope

with session_scope(config=config) as session:
    ...
```

Use this rule of thumb:

- one-off write script -> `COMMIT_ON_SUCCESS`
- read-only inspection -> `ABORT_ON_EXIT`
- library or framework code -> manual, unless you deliberately wrap it

## First Useful Persistent Write

Once login works, prove that you can store and read data:

```
from gemstone_py import GemStoneConfig, TransactionPolicy
from gemstone_py import GemStoneSession
from gemstone_py.persistent_root import PersistentRoot

config = GemStoneConfig.from_env()

with GemStoneSession(
    config=config,
    transaction_policy=TransactionPolicy.COMMIT_ON_SUCCESS,
) as session:
    root = PersistentRoot(session)
    root["DocsSmokeTest"] = {"status": "ok", "kind": "setup-guide"}

with GemStoneSession(config=config) as session:
```



```
root = PersistentRoot(session)
print(root["DocsSmokeTest"])
```

If that round trip works, the rest of the package will feel much less mysterious.

## First Async Check

If your application is async-first, run the async example after the basic sync login works:

```
python -m examples.async_features.session_root_and_collection
```

The example opens an `AsyncSession`, writes through `AsyncPersistentRoot`, keeps a managed OOP alive, and exercises `AsyncGSCollection`.

For FastAPI:

```
python -m pip install -e ".[examples]"
python -m examples.fastapi.run --reload
```

When the server starts, you should see output like:

```
INFO: Will watch for changes in these directories: ['/path/to/gemstone-py']
INFO: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO: Started reloader process [49045] using WatchFiles
INFO: Started server process [49048]
INFO: Waiting for application startup.
INFO: Application startup complete.
```

With that server running, test it from a second terminal.

Basic checks:

```
curl -i http://127.0.0.1:8000/
```

Expected:

```
HTTP/1.1 200 OK
```

Body should include:

```
{"name": "gemstone-py FastAPI example", "endpoints": {"health": "/health/gemstone", "docs": "/docs", "openapi": "/openapi.json"}}
```

Then test the GemStone endpoint:

```
curl -i http://127.0.0.1:8000/health/gemstone
```

Expected if GemStone credentials/environment are set and the stone is reachable:

```
{"result": 7}
```

Also open these in a browser:

```
http://127.0.0.1:8000/  
http://127.0.0.1:8000/docs  
http://127.0.0.1:8000/health/gemstone
```

## Typed and Lifetime Examples

Once basic reads and writes work, these examples show the newer type and export-set APIs:

```
python -m examples.typed_access.typed_oops_and_queries  
python -m examples.lifetime.managed_oop_handles
```

Use them to confirm that `TypedOop[T]`, typed `GSCollection` predicates, `execute_managed(...)`, and `session.handle(...)` behave against your stone.

## Running the Tests

Unit tests:

```
python -m unittest discover -s tests -p 'test*.py'
```

The standard local verification lane:

```
./scripts/run_ci_checks.sh
```

Live GemStone tests:

```
GS_RUN_LIVE=1 ./scripts/run_live_checks.sh
```

Longer live soak tests:

```
GS_RUN_LIVE=1 GS_RUN_LIVE_SOAK=1 ./scripts/run_live_checks.sh
```

Destructive live tests are intentionally separated. They mutate shared state and are not meant to be run casually.

## Troubleshooting

### GemStoneConfigurationError

Usually means `GS_USERNAME` and/or `GS_PASSWORD` are missing.

### OSError while loading the client library

Usually one of:

- `GS_LIB` is wrong
- `GS_LIB_PATH` points at a missing or incompatible `libgcirpc`
- the local machine does not actually have the client library installed

### Login works in one shell but not another

Check that both shells export the same environment values. This failure mode is common, boring, and surprisingly effective.

### Your writes "worked" but the data is missing

That almost always means you forgot to commit or assumed a session would commit for you. Re-check the transaction policy.

### A Flask request wrote partial data after a handled error

Use the request-session integration from `gemstone_py.web` and let request teardown own the final commit-or-abort decision. The web helpers were hardened exactly to avoid this problem.

### The native backend is not selected

Check the current process:

```
python -m examples.native_backend.check_backend
```

If `selected backend` is `ctypes`, either the optional package is not installed or `GEMSTONE_PY_GCI_BACKEND=ctypes` was set before Python started. Install `gemstone-py[fast]` and start a fresh Python process. Use `GEMSTONE_PY_GCI_BACKEND=native` when you want import failure instead of silent fallback.

## Where to Go Next

- [User Manual](#) for the main abstractions
- [Examples Guide](#) for the runnable demos
- [Cookbook](#) for copy-paste-friendly recipes

This PDF was generated locally from the Markdown sources in `docs/`. If you edit the source files, rerun `python docs/build_pdf_docs.py`.